



Specifica Tecnica

Autore Alessandro Morabito, Alessandro Frison

Verificatore Alessandro Morabito

Approvazione

Registro delle versioni

Versione	Data	Autore	Verificatore	Descrizione delle modifiche
0.3.0	25/03/2026	Alessandro Frison		Aggiunta sezione introduzione
0.2.0	25/03/2026	Lorenzo Grolla	Alessandro Frison	Stesura tecnologie utilizzate
0.1.0	24/03/2026	Alessandro Morabito	Lorenzo Grolla	Creazione documento

Indice

1	Introduzione	4
1.1	Scopo del documento	4
1.2	Glossario	4
1.3	Scopo del prodotto	4
1.4	Riferimenti	4
1.4.1	Riferimenti normativi	4
1.4.2	Riferimenti informativi	5
2	Tecnologie	5
2.1	Linguaggi di programmazione	5
2.1.1	TypeScript	5
2.2	Formato dei dati	5
2.2.1	JSON	5
2.3	Framework	5
2.3.1	NestJS	5
2.3.2	LangGraph	6
2.4	Librerie	6
2.4.1	Librerie backend	6
2.4.1.1	LangChain	6
2.4.1.2	Axios	7
2.4.1.3	dotenv	7
2.4.1.4	isomorphic-git	7
2.4.1.5	Mongoose	7
2.4.2	Librerie frontend	8
2.4.2.1	React	8
2.4.2.2	TanStack Router	8
2.4.2.3	react-markdown	8
2.5	Autenticazione	9
2.5.1	Amazon Cognito	9
2.6	Database	9
2.6.1	MongoDB	9
2.7	Testing	9
2.7.1	Jest	9
2.7.2	Vitest	9
2.7.3	Playwright	10
2.8	Infrastruttura e deployment	10
2.8.1	Docker	10
2.8.2	AWS Amplify	10
2.8.3	Amazon ECS	10
2.8.4	Amazon EC2	11

3	Architettura	11
3.1	Architettura di deployment	11
3.2	Architettura logica	11
3.2.1	Frontend	11
3.2.2	Backend	11
3.2.3	Container	11

1 Introduzione

1.1 Scopo del documento

Il presente documento si pone l'obiettivo di definire in modo esaustivo la **Specifica Tecnica** del progetto, fornendo una guida dettagliata alle scelte architetture e implementative adottate. Esso rappresenta il ponte logico tra i requisiti iniziali e l'effettiva realizzazione del software, garantendo che ogni componente sia integrata in modo coerente e scalabile.

Il documento approfondisce l'evoluzione del sistema rispetto al **Proof of Concept (PoC)** iniziale, delineando una struttura robusta e orientata alla manutenibilità a lungo termine.

Nello specifico, questo documento si propone di:

- **Delineare l'architettura di sistema:** fornendo una panoramica delle macro-componenti e del flusso di dati tra di esse;
- **Dettagliare le scelte tecnologiche:** giustificando l'adozione di specifici framework, librerie e strumenti di sviluppo;
- **Illustrare i design pattern:** descrivendo gli schemi logici e le best practice utilizzate per risolvere problematiche ricorrenti e ottimizzare la struttura del codice;
- **Definire i processi di deployment:** spiegando come le diverse componenti vengono distribuite, configurate e rese operative nell'ambiente di produzione;
- **Fornire una base per la manutenzione:** offrendo la documentazione necessaria affinché futuri sviluppatori possano comprendere ed estendere il sistema con facilità.

1.2 Glossario

Al fine di prevenire ambiguità e garantire una comunicazione uniforme e precisa tra i membri del gruppo e i committenti, è stato redatto un Glossario^G apposito. Per facilitare la lettura del presente documento, i termini tecnici o dotati di un significato specifico all'interno del dominio di progetto sono contrassegnati da una lettera «G» posta in apice (es. Parola^G). La definizione estesa di tali termini è reperibile nel documento citato tra i riferimenti informativi.

1.3 Scopo del prodotto

Il sistema CodeGuardian è progettato per condurre analisi approfondite sulla qualità di repository GitHub^G, garantendo un controllo rigoroso sulla gestione degli accessi e sull'autorizzazione all'avvio delle procedure di scansione.

La piattaforma si propone come uno strumento strategico per team di sviluppo multidisciplinari, consentendo il monitoraggio centralizzato dello stato del software e la generazione di metriche aggregate su molteplici progetti analizzati.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- Norme Di Progetto^G V1.1.0
https://byte-holders.github.io/Documentazione/RTB/Norme_Di_Progetto-V1.1.0.pdf
- Capitolato
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C2.pdf>

1.4.2 Riferimenti informativi

- Glossario V1.0.0

<https://byte-holders.github.io/Documentazione/RTB/Glossario-V1.0.0.pdf>

2 Tecnologie

Questa sezione descrive gli strumenti e le tecnologie adottati per lo sviluppo, il testing e il deployment del software relativo al progetto CodeGuardian. In particolare, vengono illustrati il linguaggio di programmazione, le librerie, i framework e le infrastrutture utilizzate.

2.1 Linguaggi di programmazione

2.1.1 TypeScript

TypeScript è un linguaggio di programmazione open-source sviluppato da Microsoft. Si tratta di un superset di JavaScript, che introduce la tipizzazione statica, consentendo di rilevare errori di programmazione in fase di sviluppo, ridurre il numero di bug e migliorare la qualità del codice.

- **Versione:** 5.x
- **Documentazione:** <https://www.typescriptlang.org/docs/>.

Nel contesto del progetto CodeGuardian, TypeScript è il linguaggio principale utilizzato per lo sviluppo sia del frontend che del backend. La tipizzazione statica è particolarmente utile in un progetto che coinvolge strutture dati complesse, come i report di analisi del codice e i grafi di scansione generati dall'agente AI.

Per il frontend si utilizza TSX, un'estensione di TypeScript che permette di definire la struttura dell'interfaccia utente in modo più leggibile e dichiarativo, integrando HTML direttamente nel codice TypeScript.

2.2 Formato dei dati

2.2.1 JSON

JSON (JavaScript Object Notation) è un formato di scambio dati leggero e indipendente dal linguaggio di programmazione. È ampiamente utilizzato nello sviluppo web e nello scambio di dati tra applicazioni. JSON è basato su due strutture di dati:

- **Oggetti:** insiemi di coppie chiave-valore;
- **Array:** liste ordinate di valori.

Nel progetto CodeGuardian, JSON viene utilizzato per la trasmissione dei dati tra frontend e backend (payload delle API REST^G), per la serializzazione dei risultati di analisi, nonché per la configurazione dell'applicazione e la memorizzazione dei dati in MongoDB.

2.3 Framework

2.3.1 NestJS

NestJS è un framework per Node.js^G che offre un'architettura modulare e scalabile per la creazione di applicazioni lato server. Si basa su Express.js e fornisce funzionalità aggiuntive come

dependency injection^G, middleware e decorator. NestJS consente di organizzare il codice in moduli riutilizzabili e di definire facilmente controller, servizi e provider.

- **Versione:** 11.x
- **Documentazione:** <https://docs.nestjs.com/>.

Nel progetto CodeGuardian, NestJS è utilizzato per creare il backend. In particolare, viene impiegato per definire i controller che gestiscono le richieste HTTP provenienti dal frontend, i servizi che implementano la business logic (gestione workspace^G, membership^G, scansioni), e i provider per l'iniezione delle dipendenze. L'architettura del backend è organizzata in sette moduli principali: WorkspaceModule, MembershipModule, RepositoryModule, ScanModule, ReportModule, AnalysisModule e GitHubModule.

2.3.2 LangGraph

LangGraph è un framework open-source per l'orchestrazione di agenti AI^G, sviluppato da LangChain Inc. Consente di modellare workflow complessi come grafi a stati, dove ogni nodo rappresenta un passo di ragionamento o un'azione, e ogni arco il flusso di controllo tra i passi. LangGraph fornisce funzionalità di gestione dello stato, checkpointing, streaming token-by-token e supporto per cicli e ramificazioni condizionali.

- **Versione:** 1.x
- **Documentazione:** <https://langchain-ai.github.io/langgraph/>.

Nel progetto CodeGuardian, LangGraph è il componente centrale dell'agente di analisi del codice. Il grafo implementato definisce il flusso di scansione dei repository GitHub^G: dalla clonazione del repository, all'analisi dei file, fino alla generazione del report finale. La struttura a grafo consente di gestire ramificazioni condizionali basate sul tipo di analisi richiesta e permette di integrare verifiche intermedie e retry automatici in caso di errore.

2.4 Librerie

2.4.1 Librerie backend

2.4.1.1 LangChain

LangChain è una libreria open-source che fornisce componenti riutilizzabili per l'integrazione di Large Language Model (LLM)^G in applicazioni software. Offre astrazioni per la gestione dei prompt, l'invocazione dei modelli, l'integrazione con tool esterni e la gestione della memoria conversazionale.

- **Versione:** 1.x
- **Documentazione:** <https://js.langchain.com/docs/>.

Nel progetto CodeGuardian, LangChain è utilizzato in combinazione con LangGraph per la costruzione dell'agente di analisi. Fornisce le astrazioni per l'invocazione dei modelli LLM e l'integrazione con i tool di analisi del codice sorgente.

2.4.1.2 Axios

Axios è un client HTTP basato su Promise per Node.js e browser. È isomorfo, cioè può essere eseguito sia nel browser che in Node.js utilizzando lo stesso codice sorgente. Lato server, Axios utilizza il modulo `http` nativo di Node.js, mentre lato client usa `XMLHttpRequest`.

- **Versione:** 1.x
- **Documentazione:** <https://axios-http.com/docs/intro>.

Nel progetto CodeGuardian, Axios viene utilizzato lato frontend per effettuare richieste HTTP al backend (interrogazione delle API REST per workspace, scansioni e report) e lato backend per comunicare con servizi esterni, come le API di GitHub.

2.4.1.3 dotenv

dotenv è una libreria che consente di caricare variabili d'ambiente da un file `.env` nell'oggetto `process.env` di Node.js. Permette di separare la configurazione dal codice sorgente, seguendo le best practice della metodologia Twelve-Factor App.

- **Versione:** 16.x
- **Documentazione:** <https://github.com/motdotla/dotenv>.

Nel progetto CodeGuardian, dotenv è utilizzato per gestire in modo sicuro le credenziali sensibili (chiavi API dei servizi LLM, credenziali MongoDB, configurazioni AWS) sia in ambiente di sviluppo che di testing, evitando di esporre dati riservati nel codice sorgente.

2.4.1.4 isomorphic-git

isomorphic-git è una reimplementazione completa di Git in puro JavaScript, compatibile sia con Node.js che con ambienti browser. Consente di eseguire operazioni Git (clone, checkout, log, status) senza dipendenze da moduli nativi C++, garantendo piena interoperabilità con l'implementazione canonica di Git.

- **Versione:** 1.x
- **Documentazione:** <https://isomorphic-git.org/>.

Nel progetto CodeGuardian, isomorphic-git è una componente fondamentale del modulo `GitHubModule`. Viene utilizzato per clonare i repository GitHub degli utenti direttamente nel backend, consentendo all'agente AI di analizzare il codice sorgente localmente senza dipendere da strumenti CLI esterni. Ciò garantisce portabilità e compatibilità con ambienti containerizzati.

2.4.1.5 Mongoose

Mongoose è una libreria ODM (Object Data Modeling) per MongoDB e Node.js. Fornisce una soluzione basata su schema per modellare i dati dell'applicazione, includendo validazione built-in, casting dei tipi, costruzione di query e hook per la business logic.

- **Versione:** 8.x
- **Documentazione:** <https://mongoosejs.com/docs/>.

Nel progetto CodeGuardian, Mongoose è utilizzato come layer di accesso ai dati per MongoDB. Tutti i modelli del dominio applicativo (workspace, membership, repository, scan, report) sono definiti tramite schema Mongoose, garantendo la validazione dei dati a livello applicativo e la coerenza delle operazioni CRUD.

2.4.2 Librerie frontend

2.4.2.1 React

React è una libreria JavaScript open-source per la creazione di interfacce utente, sviluppata da Meta. Consente di definire componenti riutilizzabili che rappresentano parti dell'interfaccia, aggiornando automaticamente la vista in base allo stato dell'applicazione. React adotta un approccio dichiarativo per la definizione dell'interfaccia utente, semplificando la gestione dello stato e la sincronizzazione con il DOM.

- **Versione:** 19.x
- **Documentazione:** <https://react.dev/learn>.

Nel progetto CodeGuardian, React è il framework di riferimento per l'interfaccia utente. La UI è organizzata in componenti funzionali che gestiscono la visualizzazione dei workspace, la lista dei repository, l'avvio delle scansioni e la consultazione dei report di analisi.

2.4.2.2 TanStack Router

TanStack Router è una libreria open-source di routing type-safe per applicazioni React. A differenza di React Router, offre un sistema di routing completamente tipizzato a livello di TypeScript, con validazione dei parametri di ricerca, caricamento dati integrato e supporto nativo per search params tipizzati.

- **Versione:** 1.x
- **Documentazione:** <https://tanstack.com/router/latest>.

Nel progetto CodeGuardian, TanStack Router gestisce la navigazione tra le diverse sezioni dell'applicazione: la dashboard iniziale, la vista dei workspace, le pagine di dettaglio dei repository e le schermate di visualizzazione dei report. L'integrazione type-safe con TypeScript garantisce che tutti i parametri delle rotte siano validati staticamente.

2.4.2.3 react-markdown

react-markdown è una libreria React che consente di renderizzare contenuto Markdown come componenti React. Supporta la sintassi CommonMark e può essere estesa tramite plugin remark e rehype per funzionalità aggiuntive (syntax highlighting, tabelle, GFM).

- **Versione:** 9.x
- **Documentazione:** <https://github.com/remarkjs/react-markdown>.

Nel progetto CodeGuardian, react-markdown viene utilizzato per visualizzare i report di analisi generati dall'agente AI, i quali sono prodotti in formato Markdown. La libreria consente di presentare all'utente il report in modo formattato, includendo intestazioni, elenchi, blocchi di codice ed eventuali tabelle riassuntive.

2.5 Autenticazione

2.5.1 Amazon Cognito

Amazon Cognito è un servizio di autenticazione e autorizzazione gestito da Amazon Web Services (AWS)^G. Consente di aggiungere funzionalità di sign-up, sign-in e gestione degli accessi alle applicazioni web e mobile. Supporta provider di identità federata (come Google e GitHub), autenticazione multi-fattore (MFA) e gestione dei token JWT^G.

- **Documentazione:** <https://docs.aws.amazon.com/cognito/>.

Nel progetto CodeGuardian, Amazon Cognito è il servizio di riferimento per la gestione dell'autenticazione degli utenti. Si occupa della registrazione, dell'accesso (anche tramite provider OAuth come GitHub), del rilascio e della verifica dei token JWT, e della gestione delle sessioni. Il backend NestJS verifica i token Cognito per proteggere le API e implementare il controllo degli accessi basato sui ruoli (owner, admin, member) all'interno dei workspace.

2.6 Database

2.6.1 MongoDB

MongoDB è un database NoSQL^G open-source, orientato ai documenti. A differenza dei database relazionali, MongoDB memorizza i dati in documenti flessibili in formato BSON (Binary JSON), consentendo strutture dati dinamiche che possono variare tra i documenti di una stessa collezione. MongoDB supporta query ricche, indicizzazione, aggregazioni e scalabilità orizzontale tramite sharding.

- **Documentazione:** <https://www.mongodb.com/docs/>.

Nel progetto CodeGuardian, MongoDB è il database principale dell'applicazione. Viene utilizzato per la persistenza di tutte le entità del dominio: workspace, membership, repository, scansioni e report di analisi. La natura document-oriented di MongoDB si adatta alla struttura dei report generati dall'agente AI, che possono variare in formato e complessità. L'accesso ai dati avviene tramite la libreria Mongoose.

2.7 Testing

2.7.1 Jest

Jest è un framework di testing progettato per garantire la correttezza di qualsiasi sorgente JavaScript. Consente di scrivere test con un'API accessibile e ricca di funzionalità, che fornisce risultati rapidamente. Supporta mocking, snapshot testing, code coverage e test paralleli.

- **Versione:** 29.x
- **Documentazione:** <https://jestjs.io/docs/getting-started>.

Jest è utilizzato per eseguire i test di unità e di integrazione del backend, verificando che i controller, i servizi e i moduli NestJS siano implementati correttamente. In particolare, viene impiegato per testare la logica di business dei sette moduli dell'applicazione.

2.7.2 Vitest

Vitest è un framework di testing per Vite. Offre funzionalità avanzate come il supporto nativo per TypeScript, la configurazione tramite file di setup e la possibilità di eseguire test in modalità

watch. Questa modalità è particolarmente utile durante lo sviluppo attivo e il debugging, poiché riduce i tempi di attesa per l'esecuzione dei test.

- **Versione:** 3.x
- **Documentazione:** <https://vitest.dev/guide/>.

Vitest è utilizzato per eseguire i test di unità e di integrazione del frontend, verificando che i componenti React e le funzionalità di interfaccia siano implementati correttamente.

2.7.3 Playwright

Playwright è un framework di testing end-to-end per applicazioni web, sviluppato da Microsoft. Consente di scrivere test in JavaScript/TypeScript, eseguirli in browser reali (Chromium, Firefox, WebKit) e verificare il comportamento dell'applicazione in modo automatizzato.

- **Versione:** 1.x
- **Documentazione:** <https://playwright.dev/>.

Playwright è utilizzato per testare il frontend dell'applicazione end-to-end, verificando che i flussi utente principali (registrazione, login, creazione workspace, avvio scansione, visualizzazione report) funzionino correttamente attraverso l'intero stack applicativo.

2.8 Infrastruttura e deployment

2.8.1 Docker

Docker è una piattaforma open-source per lo sviluppo, la distribuzione e l'esecuzione di applicazioni in contenitori. I contenitori Docker sono unità di software che includono il codice sorgente, le dipendenze e le configurazioni necessarie per eseguire un'applicazione, garantendo ambienti consistenti e facilmente riproducibili.

- **Documentazione:** <https://docs.docker.com/>.

Nel progetto CodeGuardian, Docker è utilizzato per containerizzare i singoli servizi dell'applicazione (frontend, backend, agente AI) durante lo sviluppo e il testing locale, garantendo coerenza tra gli ambienti di sviluppo e produzione.

2.8.2 AWS Amplify

AWS Amplify è un servizio di hosting e deployment gestito da Amazon Web Services, progettato per applicazioni frontend e web full-stack. Supporta il deployment continuo da repository Git, la gestione automatica di certificati SSL, CDN globale e preview per le pull request.

- **Documentazione:** <https://docs.amplify.aws/>.

Nel progetto CodeGuardian, AWS Amplify è utilizzato per il deployment e l'hosting del frontend React. Fornisce un ambiente di hosting scalabile con deployment continuo collegato al repository GitHub del progetto.

2.8.3 Amazon ECS

Amazon Elastic Container Service (ECS) è un servizio di orchestrazione di contenitori completamente gestito da AWS. Consente di eseguire, fermare e gestire contenitori Docker su un

cluster di istanze EC2 o tramite AWS Fargate (modalità serverless). ECS supporta la scalabilità automatica, il bilanciamento del carico e l'integrazione nativa con altri servizi AWS.

- **Documentazione:** <https://docs.aws.amazon.com/ecs/>.

Nel progetto CodeGuardian, Amazon ECS è utilizzato per il deployment e l'orchestrazione dei container che eseguono il backend NestJS e il servizio dell'agente AI basato su LangGraph. ECS garantisce alta disponibilità e scalabilità dei servizi backend in ambiente di produzione.

2.8.4 Amazon EC2

Amazon Elastic Compute Cloud (EC2) è un servizio di cloud computing che fornisce capacità computazionale ridimensionabile nel cloud AWS. Consente di lanciare istanze di server virtuali con diversi sistemi operativi e configurazioni hardware.

- **Documentazione:** <https://docs.aws.amazon.com/ec2/>.

Nel progetto CodeGuardian, le istanze EC2 costituiscono l'infrastruttura computazionale su cui vengono eseguiti i container gestiti da Amazon ECS. La scelta del tipo di istanza è calibrata in base ai requisiti di memoria e CPU necessari per l'esecuzione delle analisi AI.

3 Architettura

3.1 Architettura di deployment

3.2 Architettura logica

3.2.1 Frontend

3.2.2 Backend

3.2.3 Container